

UNIT IV
TURING MACHINE

Contents

1. Turing machines	2
2. Computable languages and functions	6
3. Variations of TMs	7
4. Recursive and recursive enumerable languages	8

UNIT IV

Turing machines: Turing machines(TM) – computable languages and functions – Turing machine constructions – storage in finite control - variations of TMs – recursive and recursive enumerable languages.

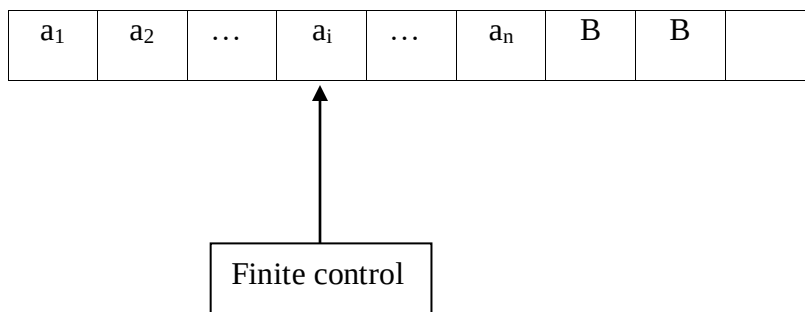
1. TURING MACHINES

It is a more powerful model than many models. It was introduced by Alan Turing in 1936. It can do everything that a real computer can do.

Configuration of the TM

It has a finite control, an input tape that is divided into cells, each of which is capable of holding one symbol. Associated with the tape head is a read write head [tape head] that can

- Travel right or left on the tape
- Scan one cell of the tape at a time.
- Read and write a single symbol on each move.



The TM's temporary storage is on the tape. Whatever input and output is necessary will be done on the tape. The tape has a leftmost cell but is infinite to its right. The tape head is initially set to scan the content of the leftmost cell.

Difference between FA and TMs

1. A TM can both write on the tape and read from it.
2. The read write head can move both left and to the right.
3. The tape is infinite.
4. The special states for rejecting and accepting take immediately effect.

Moves of the TM

Depending on the state of the infinite control the symbol on the cell scanned by the tape head, TM changes state, prints a new symbol on the tape cell scanned and moves left or right or stationary.

Mathematical definition of a TM

The basic model of a TM is defined as

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where,

Q – a finite set of states.

Γ – a finite set of allowable tape symbols.

B – a symbol of Γ , is the blank

Σ - a subset of Γ not including β is the set of input symbols.

δ – the next move function, (if) transition function), from $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$

[δ may not be defined for some values]

q_0 – it is in Q , it is the start state

F - (Subset of Q) is the set of accepting states.

Varieties of Turing Machine (Types or modified versions of TM)

The basic model of the Turing Machine is equivalent to many other modified versions. They are

1. Non deterministic Turing machine
2. Two way infinite tape.
3. Multi tape Turing machine
4. Offline Turing machine
5. Multi head Turing machine
6. Multidimensional Turing machine
7. Restricted Turing machine

1. Non Deterministic Turing machine

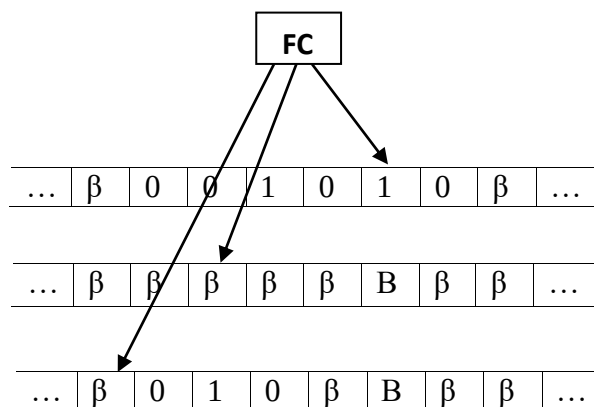
- It has a finite control and a single one way infinite tape.
- For the given state and the tape symbol scanned by the tape head the TM has a finite number of possibilities for its next move.
- Here the computation is like a tree whose branches correspond to different possibility.
- If some branch of the computation leads to the accept state the machine accepts the input.

2. Two way Infinite Turing machine

- This is similar to the basic model.
- But here the tape is infinite to the left as well as to the right.
- Infinite number of blanks will be in both sides.
- The initial ID is q_0w

3. Multi tape Turing machines

- It has a finite control with many tapes say 'k' tape heads.
- Each tape is two way infinite.
- Initially the input will be on the first tape and the other tapes are kept blank.
- Depending on the states of the PC and the symbol scanned by each of the tape heads the TM can
 - Change the state
 - Print a new symbol on each of cells scanned by the tape heads.
 - Move each tape head independently [some to right, some to left and some other stationary]



4. Offline Turing Machine

- This is a multi tape TM whose input tape is read only.
- The input is surrounded by some end markers.
- The TM is allowed to move the input head in between the end markers.

5. Multi head Turing machine

- It has 'k' heads, $k > 1$.
- Depending on the state and the symbol scanned by each head, the heads move independently.

6. Multidimensional Turing machine

- It has a finite control and a tape consisting of k-dimensional arrangement of the cells infinite in all directions for some 'k'.
- Initially the input string is along one axis and the tape head is input string.
- Depending on the state and the symbol scanned the TM
 - i. Changes state
 - ii. Prints a new symbol.
 - iii. Moves the tape head in one of the 2k direction positively or negatively along the k-axes.

7. Restricted Turing machine

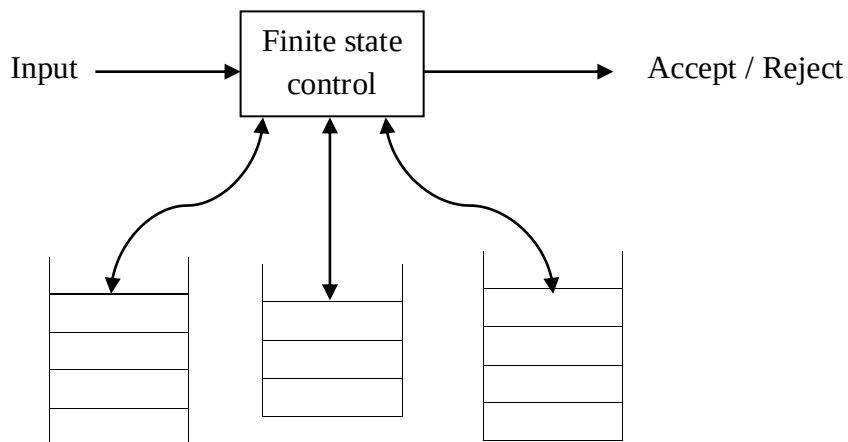
- i. Counter machines
- ii. Multi stack machines
- iii. Turing machine with semi infinite tapes.

(i) Counter machine

- It has the same structure as the multistack machine.
- It has ability to store a finite number of the integers. (counters)
- It can able to add or subtract one from the counter but it cannot tell two different nonzero counts from each other.

(ii) Multistack machine

- A k stack machine is a deterministic PDA with k stacks.
- It has a Finite control which is in one of the state.
- It has a finite stack alphabet which is used for all stacks.
- A move of the multistack machine is based on
 - a. The state of the finite control.
 - b. The input symbol read.
 - c. The top stack symbol on each stack.



A multi stack machine with 3 stacks

In one move the multi stack machine can

- (i) Change to a new state
- (ii) Replace the top symbol of each with a string or more stack symbols.

(iii) Turing machine with semi infinite tapes

- Here the tape is semi infinite that is, there are no dells to the left of the initial position of the tape head.
- It can simulate one whether is infinite in both directions.
- It accepts any regular language.

Universal Turing machine [UTM]

It is a programmable computer. Since when it is given a program a description of another computer it makes itself as if it were that machine while processing the input.

2. COMPUTABLE LANGUAGES AND FUNCTION:

A language that is accepted by a Turing machine is said to be *recursively* enumerable (r.e). The term “enumerable” derives from the fact that is precisely these languages whose strings can be enumerated (listed) by a TM. “Recursively” is a mathematical term predating the computer, and its meaning is similar to what the computer scientist would cal “recursion”. The class of r.e is very broad and properly includes the CFL’s.

The class of r.e languages includes some languages for which we cannot mechanically determine membership. If $L(M)$ is such a language, then any Turing machine recognizing $L(M)$ must fail to halt on some input not in $L(M)$. If w is in $L(M)$, M eventually halts on input w . However, as long as M is still running on some input, we can never tell whether M will run forever.

It is convenient to single out a subclass of the r.e. sets called the recursive sets, which are those languages accepted by a least one TM that halts on all inputs (note that halting may or may not be preceded by acceptance).

The Turing Machine as a computer of integer functions:

In addition to being a language acceptor, the TM may be viewed as a computer of functions from integers to integers. The traditional approach is to represent integers in unary; the integer $i \geq 0$ is represented by6 the string 0^i . If a function has k arguments, $i_1, i_2, \dots, i_k$, then these integers are initially placed on the tape separated by 1’s, as $0^{i_1}10^{i_2}1 \dots 10^{i_k}$.

If the TM halts (whether or not in an accepting state) with a tape consisting of 0^m for some m , then we say that $f(i_1, i_2, \dots, i_k) = m$, where f is the function of k arguments computed by this TM. Note that one TM may compute a function of one argument, a different function of

two arguments, and so on. Also note that if TM M computes function f of k arguments, then feed not have a value for all different k – tuples of integers $i_1, \dots, i_k$.

If $f(i_1, \dots, i_k)$ is defined for all $i_1, i_2, \dots, i_k$, then we say f is a total recursive function. A function $f(i_1, i_2, \dots, i_k)$ computed by a Turing Machine is called a partial recursive function. In a sense, the partial recursive functions are analogous to the r.e. languages, since they are computed by Turing Machine that may or may not halt on a given input. The total recursive functions correspond to the recursive languages, since they are computed by TM's that always halt. All common arithmetic functions on integers, such as multiplication, $n!$, $\lfloor \log_2 n \rfloor$ and 2^n are total recursive functions.

3. TECHNIQUES FOR TURING MACHINE CONSTRUCTION

Designing Turing machines by writing out a complete set of states and a next – move function is a noticeably unrewarding task. In order to describe complicated Turing machine constructions we need some “higher – level” conceptual tools.

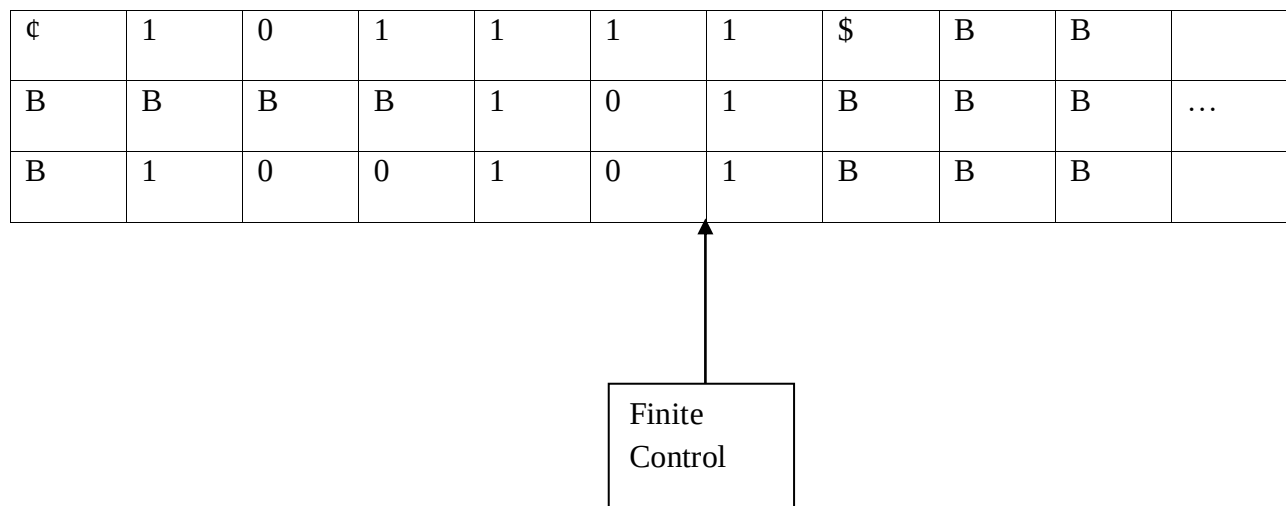
Storage in the finite control

The finite control can be used to hold a finite amount of information. To do so, the state is written as a pair of elements, one exercising control and the other storing SYMBOL. It should be emphasized that this arrangement is for conceptual purposes only. No modifications in the definition of the Turing machine have been made.

In general, we can allow the finite control to have k components, all but one of which store information.

Multiple tracks

We can imagine that the tape of the Turing machine is divided into k tracks, for any finite k . This arrangement is shown in Fig. with $k=3$. The symbols on the tape are considered k – tuples, one component for each track.



Checking off symbols

Checking off symbols is a useful trick for visualizing how a TM recognizes languages defined by repeated strings, such as

$\{ww \mid w \in \Sigma^*\}$, $\{wcy \mid w \text{ and } y \in \Sigma^*, w \neq y\}$ or $\{ww^R \mid w \in \Sigma^*\}$.

It also useful when lengths of substrings must be compared, such as in the languages

$\{a^ib^i \mid i \geq 1\}$ or $\{a^ib^jc^k \mid i \neq j \text{ or } j \neq k\}$

Shifting over

A Turing machine can make a space on its tape by shifting all non blank symbols a finite number of cells to the right. To do so, the tape head makes an excursion to the right, repeatedly storing the symbols read in its finite control and replacing them with symbols read from dells to the left. The Tm can then return to the vacated dells and print symbols of its choosing. If space is available, it can push blocks of symbols left in a similar manner.

Subroutines

As with programs, a “modular” or “top-down” design is facilitated if we use subroutines to define elementary processes. A Turing machine can simulate any type of subroutine found in programming languages, including recursive procedures and any of the known parameter-passing mechanisms.

The general idea is to write part of a TM program to serve as a subroutine; it will have a designed initial state and a designated return state which has no move and which will be used to effect a return to the calling routine. To design a TM that “calls” the subroutine, a new set of states for the subroutine is made, and a move from the return is effected by the move from the return state.

4. RECURSIVE & RECURSIVE ENUMERABLE LANGUAGES

1. Recursive enumerable language:

A language L is called a recursive enumerable language if

- (i) There is a TM that accepts L and
- (ii) Rejects L' or goes into an infinite loop

i.e. The TM can halt in the accepting state but need not halt in the non accepting state.

2. A language L is said to be recursive if

- (i) There is a Turing machine that accepts L and
- (ii) Rejects L'

i.e. Here the TM can halt (1) in the accepting states and (2) in the non accepting state.

A recursive language is recursive enumerable but not the converse.

i.e. A recursive language need not be recursive.

Properties of a recursive or recursive enumerable language:

Result 1:

If a language is recursive then it is recursive enumerable.

Result 2: Recursive languages are closed under complementation.

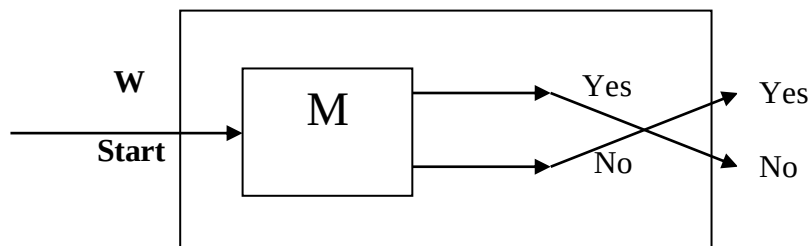
i.e. If L is recursive then its complement is recursive

To prove: The complement of L say L' is recursive

Proof: Let L be a recursive language. Then there is a Turing Machine M that accepts L .

Construct a Turing machine M' from M such that : M enters a final state on the input W in L , then M' halts in a non accepting state. (Without accepting)

If M halts without accepting then M' accepts the input ' w ' in L' . Hence M' is an algorithm and is given as below.



Here 'Yes' corresponds to 'accept' and 'No' corresponds to 'Reject'. The language of M' is $L(M')$ and is the complement of L . Hence the proof.

Result 3: The union of two recursive language is recursive.

To prove: $L_1 \cup L_2$ is recursive.

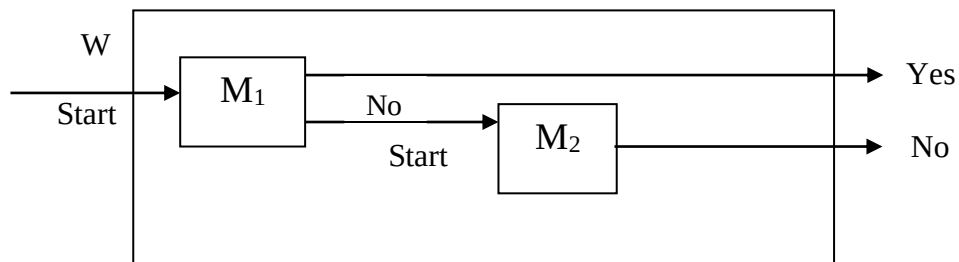
Proof: Let L_1 and L_2 two recursive languages. Then there exists Turing machine M_1 and M_2 such that M_1 and M_2 accept L_1 and L_2 respectively.

Construct a Turing machine M that accepts $L_1 \cup L_2$ as below:

- First simulate M on M_1 .
- If M_1 accepts inputs w in $L_1 \cup L_2$ then M accepts.
- If not, M simulates M_2 and accepts iff M_2 accepts.

Since M_1 and M_2 are algorithms (used to say Yes or No), M_1 and M_2 will not hang, they will halt (in accepting state or non accepting state). M will surely halt (It will not hang).

Hence M is an algorithm. M is given below:



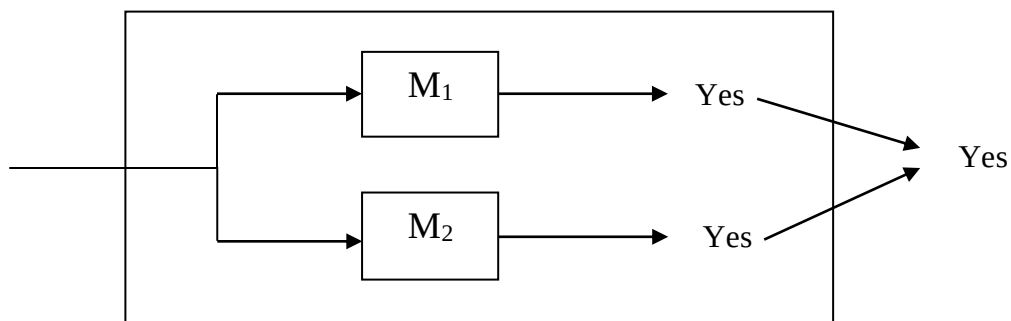
Now the language of M is $L_1 \cup L_2$. Hence the result.

Result 4: The union of two recursively enumerable languages enumerable.

Proof: Let L and L_2 be two recursively enumerable languages.

There are Turing machine M_1 and M_2 such that M_1 accepts L_1 and M_2 accepts L_2 . Construct a Turing machine M that accepts $L_1 \cup L_2$ as below:

The above procedure will not work here. Since M_1 may not halt for some input W in L_1 . Hence M is constructed as below:



Now the language of M is $L_1 \cup L_2$. Hence $L_1 \cup L_2$ is recursively enumerable.

Result 5: If a language L and its complement L' are both recursively enumerable, then L (and hence L' by **result 2**) is recursive.

To prove: L is recursive

Proof: Let a language L and its complement L' are recursively enumerable. Then there are Turing machine M_1 and M_2 that accepts L and L' respectively.

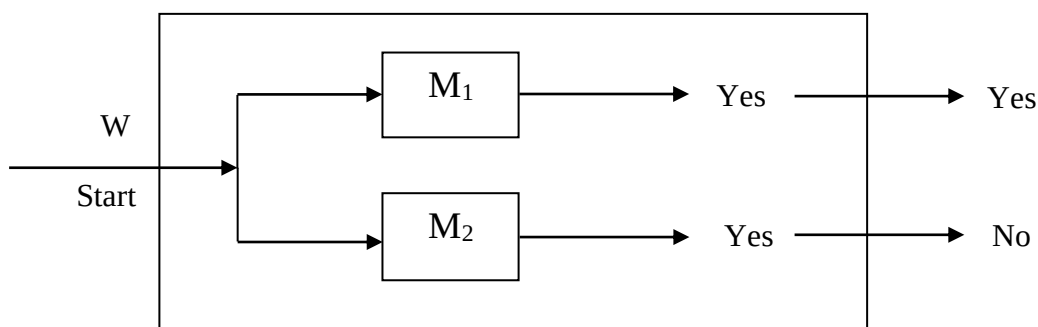
Construct a Turing machine M that accepts L and say either 'Yes' or 'No' as below: M is connected in such a way that M simulate M_1 and M_2 simultaneously.

- If M_1 accepts then M will accept it.
- If M_2 accepts then M will reject it.

Hence M will always say either 'yes' or 'no' and not both.

Hence M is an algorithm that accepts that accepts L . Hence L is recursive. {Proved Ans (1) }
Hence L' is recursive. {Ans (2)}

M is shown below:



Hence we have If L and L' are a pair of complementary languages then either

- i. Both L and L' are recursive (or)
- ii. Neither L nor L' are recursively enumerable.